

CS 320 Project Two: Summary and Reflections

Report

Summary: Alignment to Requirements

The customer requirements were translated directly into JUnit test cases for the Contact, Task, and Appointment services. The completed project contains 50 JUnit tests that verify object creation, field validation, update operations, deletion operations, uniqueness requirements, boundary conditions, and exception handling. Each requirement is mapped to one or more test cases, providing measurable evidence that the software satisfies the functional requirements established by the customer.

Summary: Effective Tests

The JUnit framework provides assertions that allow developers to automatically verify expected outcomes and detect defects early in the development process (JUnit Team, 2025). The project uses `assertEquals()`, `assertNotNull()`, `assertThrows()`, and `assertTrue()` to evaluate both valid and invalid behaviors. These assertions function as testing oracles because they define the expected result before execution and compare expected and actual outcomes automatically.

Summary: Technically Sound Code

Technical soundness is demonstrated through defensive programming, validation controls, and exception handling. Invalid data is rejected before it can corrupt application state, and tests verify that exceptions are thrown when requirements are violated. Defensive-copy testing confirms that mutable appointment dates cannot be modified through external references. According to Oracle (2025), defensive copying helps preserve object integrity and supports reliable object-oriented design.

Summary: Efficient Code

The project follows a service-based architecture in which `ContactService`, `TaskService`, and

AppointmentService manage business logic separately from domain objects. This organization reduces coupling, improves maintainability, and simplifies debugging. Each JUnit test focuses on a single requirement, allowing failures to be isolated quickly. The structure promotes readability, maintainability, scalability, debugging efficiency, and long-term software quality.

Testing Results

The completed project executed 50 JUnit tests with a 100% pass rate. The test suite verified object creation, field validation, update operations, deletion operations, uniqueness requirements, boundary conditions, exception handling, and defensive programming controls across the Contact, Task, and Appointment services.

Coverage analysis was performed using EcEmma. The project achieved high coverage across the Contact, Task, and Appointment classes by testing both valid and invalid inputs, boundary values, exception conditions, and service-layer operations. Coverage analysis confirmed that major requirements, validation paths, and critical business logic were exercised during testing. Exact coverage metrics are available in the EcEmma report generated during testing.

Reflection

The project employed unit testing, boundary-value analysis, positive testing, negative testing, exception testing, and defensive-copy testing. Additional testing approaches such as integration testing, system testing, performance testing, security testing, and user acceptance testing could further strengthen software quality in a production environment.

The primary testing oracle for this project consisted of customer requirements translated into assertions. When actual behavior matched expected behavior, tests passed; otherwise, tests failed. This approach reduced the likelihood of undetected defects and increased confidence that the software satisfied customer expectations.

Caution was demonstrated by intentionally testing null values, duplicate identifiers, oversized fields, invalid dates, and invalid input conditions. Bias was reduced by attempting to break the software rather than only verifying successful outcomes. Discipline was demonstrated through systematic validation of every requirement, exception condition, and boundary case across the Contact, Task, and Appointment services.

References

JUnit Team. (2025). JUnit 5 User Guide. Retrieved June 5, 2026, from <https://junit.org/junit5/docs/current/user-guide/>

Oracle. (2025). Java Platform, Standard Edition Documentation. Retrieved June 5, 2026, from <https://docs.oracle.com/en/java/>